Paper III: Formal proof supplement

Juan Pablo Traverso Gianini
Independent researcher, Santiago, Chile
jtraverso@gmail.com
ORCID: 0009-0003-6068-4096

Preprint 1.0 — 1 November 2026. Formal verification: PASS.

1. Scope of the proof

The complete general proofs are the Lean declarations below. The manuscript explains their construction, quantitative interfaces and assembly, and gives the application proofs. The local bank, packing and Pasch insertion lemmas are supplied in the formal supplement; the paper does not claim to reproduce them all as a self-contained prose proof.

The two-scale Latin application includes the simultaneous coarse-block constraints and the count-gap corollary. Its formal proof uses the diagonal bound (8.24); the exact finite identity (8.23) is a separate written calculation. The height program has 312 arithmetic definitions and an audited equality with the height used in Theorem 7.2.

2. Audited declarations

Result	Declaration
T	`TypedTriangle.Vortex.main_theorem_T`
T+	`TypedTriangle.Vortex.main_theorem_T_with_triangle_monitors_and_pasch`
7.2	`TypedTriangle.Vortex.Explicit.main_theorem_T_with_triangle_monitors_at_tower_order`
8.1	`TypedTriangle.Applications.Latin.latin_square_with_profile_statistics`
8.2	`TypedTriangle.Applications.Latin.uniform_with_intercalates`
8.3	`TypedTriangle.Applications.Latin.signed_profile_with_intercalates`
8.4	`TypedTriangle.Applications.Latin.TwoScale.latin_two_scale_with_blocks`
8.4	`TypedTriangle.Applications.Latin.TwoScale.intercalate_gap_of_targets`
8.5	`TypedTriangle.Applications.FivePartite.main_theorem`
H	`TypedTriangle.PrintedHeight.height_agrees`

3. Source identity and axioms

The source supplement contains 1,141 project modules. The completed reconstruction records successful compilation of all of them. Fifteen source files then received comment-only documentation corrections: their noncomment text was checked for exact equality and those modules were recompiled. The other

1,126 compiled modules were reused after hash validation. All six endpoint axiom queries were rerun successfully against this object collection. H1–H4 and the mathematical Lean code were unchanged.

Only the following foundational axioms occur: `propext`, `Classical.choice`, `Quot.sound`. The queries use `pp.fullNames=true`; the permitted list contains no project-specific axioms or `sorryAx`.

Source archive: `PAPER_III_LEAN_SOURCE_v1.0.zip`.

SHA-256: `a5fbd7af2d13594a894d61bde81126353bd7d186f335ad9f302a5fd7ba942d95`.

Source manifest SHA-256:
`16e579200583065b7f5ba46db2c4f8af84608c551f5c736834abcc044286dc37`.

4. Reproduction

Use Lean 4.35.0-rc3 and Mathlib revision `c55e6e786f49471c72fbddbec5415808896aec1e`. Verify the archive and its internal manifest, then build the eight explicit targets:

```
lake build RequestProject.TypedTriangle.MainPaperIII
lake build RequestProject.TypedTriangle.MainPaperIIIPasch
lake build RequestProject.TypedTriangle.MainPaperIIITriangleTower
lake build RequestProject.TypedTriangle.ApplicationsLatinMain
lake build RequestProject.TypedTriangle.ApplicationsLatinCorollaries
lake build RequestProject.TypedTriangle.ApplicationsFivePartiteMain
lake build RequestProject.TypedTriangle.ApplicationsTwoScaleFinal
lake build RequestProject.TypedTriangle.PaperIIIHeightProgram
```

The default Lake target reaches T only; compilation of the other entries is not inferred from it. Run the six files in `audit/` separately with `lake env lean -Dpp.fullNames=true`. Their outputs and source bindings accompany the supplement. Dependencies are pinned and must not be updated during reproduction.

The recorded reconstruction reused validated compiled objects, including complete Lean object families. It is not a fresh build from an empty cache and did not rebuild external dependencies. The source supplement contains no binary cache. Its README gives the full reproduction commands and explains the separation of compatible audit import closures.

The files `HEIGHT_PROGRAM_en.md` and `HEIGHT_PROGRAM_es.md` print identical arithmetic code. They specify a threshold for triangle statistics; no explicit joint Pasch threshold is asserted.

5. Map of the construction

The following files identify the technical construction used in Sections 3–6. They belong to the source closure of T or T+ as appropriate. The explicit-order counterparts belong to the quantitative argument in Section 7; their use is not inferred merely from the default build target.

Manuscript step	Formal source
Local quotient and vertex-deletion persistence	`VertexFirstEntry.lean`, `PreparedCore Construction.lean`
Sparse edge-deletion persistence	`SparseEdgeHost.lean`, `G4ReferenceH2Generic. lean`
Smooth inverse on the paid child and exactification	`VortexPaidChildSmooth.lean`, `Smooth Exactification.lean`
Raw profile through the prepared chain	`VortexIntrinsicRawPathsPreparedVortex. lean`
Reserve chosen uniformly before the residual	`VortexUniformTerminalReserve.lean`
Signed tests with a lower envelope mass	`SignedEnvelopePacking.lean`
Initial Pasch budget and outer rate	`PaschAdaptiveInitialState.lean`, `PaschFinal AssemblyParameters.lean`
Pasch insertion bounds and inner recurrence	`PaschConcreteInnerAdaptiveLayers.lean`
Assembly of T and T+	`MainPaperIII.lean`, `MainPaperIIIPasch.lean`

The internal module-map certificate records import reachability and exact declarations. In particular, the exactification input must lie in the real incidence image, the signed-envelope theorem requires linear

lower mass, and the Pasch estimates use the available-family codegree bounds. The manuscript states these interfaces; the source files contain their complete local constructions.

The reference numbers [16] and [17] in the tool disclosure below refer to the bibliography of the accompanying manuscript.

Acknowledgments

The author is deeply grateful to his wife María Paz and his children Lucas, Juan Cristóbal, Francisca, Raimundo and Benjamín for their love, patience and support.

Use of artificial intelligence and computational tools

Claude, by Anthropic, and ChatGPT/Codex, by OpenAI, were used to explore and check arguments and to prepare the manuscript. Aristotle, by Harmonic, contributed to exploring approaches, searching for counterexamples, developing proofs, and formalizing and reviewing them in Lean. Its role was not limited to translating finished proofs.

Certo [16] and Jacobian [17] were used to explore finite instances, check identities and assess candidate constructions. These computations informed the development of the arguments; finite checks were not treated as proofs of universal statements.

The author remains responsible for the arguments, citations, code and presentation. No AI system is listed as an author.